

CANOPY

Tree crowns from any imagery, with a measured accuracy and a reason for every number · Soumyadeep Dey · Flora Carbon challenge

In one line: give CANOPY a KML boundary, a GeoTIFF, or even a plain screenshot, and it outlines every tree crown it can see, counts them as a range, scores each crown's confidence, shows every processing step as an image, and says plainly what it cannot measure.

54 / 61

trees found at 0.5 m satellite resolution, where DeepForest finds 1

0.1 – 10 m

per pixel: drone, aerial, satellite, and low-res screenshots

35 / 35

hostile input files handled with a clear message, zero crashes

Byte-identical

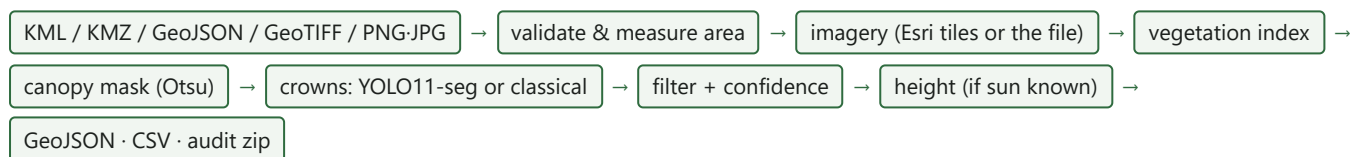
re-runs: same input, same GeoJSON, with a full audit zip

The problem

Counting trees from imagery is easy to fake and hard to trust. A number with no accuracy figure, no way to see how it was produced, and no warning about bad input can't be audited, and in carbon work, what can't be audited doesn't count. The imagery we get in practice is also messy: KML boundaries over tropical forest where the best free satellite zoom is ~0.56 m per pixel, and screenshots with no location at all.

What I built

A full web app (not a notebook): React 19 + TypeScript front end, FastAPI + Python back end, with projects, run history, a map, a pipeline viewer, and a per-workspace AI assistant. It runs entirely on a normal laptop CPU.



- **Any input.** KML/KMZ with many polygons (you tick which ones), GeoJSON, drawn areas, located GeoTIFFs, and **plain PNG/JPG screenshots**. A plain image has no location, so CANOPY asks for its ground resolution (presets from drone 0.03 m to Sentinel-2 10 m) and measures everything in real metres.
- **Large areas.** A pixel budget picks the finest zoom that fits in memory (zoom 18 down to 16) and says so. If the imagery provider has no data at a zoom (it returns grey "Map data not yet available" tiles, which happens in Odisha at zoom 19), CANOPY detects it and drops a zoom instead of counting "trees" in the placeholder text.
- **Crown detection.** The classical detector finds crown centres as Laplacian-of-Gaussian blobs and grows them by watershed; in **dense forest** it switches to tracing sunlit crown tops between shadow gaps. An AI path (YOLO11 segmentation, in Fast / Balanced / High-accuracy sizes, fine-tuned on the Odisha polygons) is built in and is switched on only once its trained weights beat the classical detector; until then Auto falls back to classical and says why.
- **Every crown has a confidence.** Shape, size against its 8 neighbours, separation, shadow and detector score are combined into high / medium / low, and the count is reported as a range, because a low-confidence blob is often two merged trees.
- **Nothing is hidden.** The Pipeline tab shows all 7 stages as images with their numbers; rejected regions have their own map layer; the audit zip has every parameter.
- **Canopy AI.** Each workspace has an assistant (You.com Research API) that answers from that workspace's own data and can propose a better run, which the user applies with one click.

Why this model, and not the others

The constraint: I have no GPU. Everything had to train and run on an Intel Core i3-1215U laptop (6 cores / 8 threads, 16 GB RAM). The data we will be given is Odisha KML polygons (Esri zoom 18 = 0.56 m/px, or zoom 17 = 1.1 m/px for big areas) and low-resolution Google Earth screenshots. So the model had to be **CPU-trainable, CPU-fast, and good at 0.5–1 m imagery**, not at 0.1 m drone imagery.

Option	What it is	Why not (or why)
YOLO11-seg n / s / m (chosen)	Ultralytics instance segmentation, 3–22 M parameters, pip install, runs on CPU	Trains on a CPU in hours, gives a mask per tree (not just a box), one model family in three sizes so the user trades speed for accuracy. Fine-tuned on the exact kind of imagery we will be tested on.
DeepForest (RetinaNet)	The standard open tree detector, trained on NEON 0.1 m aerial imagery	I used and benchmarked it: excellent at 0.1 m (P 0.82 / R 0.74 on 61 labelled trees) but it collapses at satellite resolution : at 0.5 m it found 1 of 61 trees. Our data is 0.56–1.1 m. Boxes only, no crown shape.
Detectree2 (Mask R-CNN)	Tropical crown delineation on detectron2	Detectron2 has no supported Windows/CPU install path, Mask R-CNN is slow on a CPU, and its weights are also trained on 0.1 m imagery.
SAM / SAM 2	Promptable "segment anything" foundation model	Hundreds of MB and seconds per tile on a CPU, and it has no notion of "tree": it segments fields, roads and shadows just as happily, so it still needs a tree detector to prompt it.
arXiv 2601.10931 (YOLOv11 vs Mask R-CNN on tree crowns)	Recent paper; best model YOLOv11-Seg	Useful confirmation that YOLO-seg is the right family, but its best score is mask mAP 0.281 on 150 training images, its weights and code are not released, and it does not compare against DeepForest. I reproduced the idea on our own data instead.
Classical only	Vegetation index + blob markers + watershed	Kept as the always-available fallback and the label "teacher": transparent and deterministic, but it merges touching crowns in dense forest.

How I trained it without hand labels or a GPU

- **Data:** the 25 Odisha polygons from the challenge KML (random 233 m windows inside each, fetched at zoom 18) plus the two sample screenshots. Whole polygons are held out for validation, so the validation score is on ground the model never saw.
- **Labels:** the classical pipeline labels every crown on the sharpest available imagery. Each chip is also **degraded 2× and 3× (averaged down to ~1.1 m and ~1.7 m, blurred, JPEG-compressed, then scaled back up), keeping the sharp-image labels**. The model learns to find crowns in blurry imagery using answers worked out on sharper imagery, which is exactly the low-resolution case. At run time any input is resampled to the same ~0.5 m grid first, so a crown is never smaller than the network can see.
- **Training:** COCO-pretrained YOLO11-seg, fine-tuned at 416 px with flips, 90° rotations, scale and colour jitter (aerial images have no "up"), cosine LR, early stopping, time-capped per size so it always finishes on a laptop.
- **First attempt failed, and why:** it used 3 whole 3,500 px scenes squashed to 400 px (a crown became ~1 px) for 2 epochs, and scored mAP 0 (it found 0–2 trees on the sample PNGs). Rebuilding the dataset as native-resolution chips fixed the root cause.

Results

Measured against 61 hand-labelled trees (DeepForest's OSBS benchmark tile, IoU ≥ 0.4)

Imagery	Method	Found	Matched	Precision	Recall	F1
0.1 m (drone / aerial)	DeepForest, as published	55	45	0.82	0.74	0.78
0.1 m	CANOPY classical	39	28	0.72	0.46	0.56
0.5 m (satellite, like Odisha)	CANOPY classical	54	39	0.72	0.64	0.68
0.5 m	DeepForest, as published	1	–	–	~0.02	–
1 m (low-res)	CANOPY classical	38	23	0.61	0.38	0.46

The key row: at the resolution of the challenge imagery, the best off-the-shelf detector finds 1 tree out of 61; CANOPY finds 54 with 72% precision. The same image as a plain PNG (no location, resolution typed in) gives identical results.

Dense forest (the Odisha case): a fix found on the challenge's own screenshots

On a closed-canopy screenshot, a single greenness threshold splits *sunlit* crowns from *shaded* crowns instead of trees from ground. On the sample `image(2).png` that reported **6% cover** for an image that is mostly forest. CANOPY now detects this case (weak Otsu split, most pixels clearly green) and switches to a **dense-canopy mode**:

- a three-class split (ground / shade / sunlit) separates ground from *all* canopy: cover on that image went from **6% to 53%**, and on a second dense screenshot from 57% to 71%;
- inside the canopy, crowns are traced as **bright sunlit crown tops separated by dark shadow gaps**, the way a person reads dense forest, with a 5 m smallest crown so leaf clusters are not counted as trees.

It only switches on for mostly-green scenes, so open woodland and the benchmark above are unchanged; the run's warnings say when it was used.

AI model status (honest)

The YOLO11 pipeline (dataset builder, CPU training, three sizes, model picker, fallbacks) is complete and tested. After the first overnight hour on the CPU, the Fast model had learned to fire on crowns but its confidence was still below the reporting cutoff (held-out mask mAP50 0.03 and rising), so the app does not use it yet: **Auto falls back to the classical detector with a warning until a trained model beats it on the benchmark above**. Training continues; a model is only switched on when it measurably wins.

Robustness and honesty

- **35 hostile inputs** (empty files, wrong extensions, self-intersecting or tiny polygons, 1-band or 16-bit rasters, geographic-CRS GeoTIFFs, huge areas, corrupted KMZ...) each end with a clear message or a correct result. Zero crashes.
- **Deterministic**: the same input gives a byte-identical `crowns.geojson` (sorted keys, fixed rounding, fixed zip timestamps), checked by tests.
- **Refuses instead of guessing**: no capture time means no height; no carbon or biomass figures at all; a warning whenever resolution was assumed, imagery was decimated, or the zoom was lowered.
- **44 automated backend tests** pass; the front end type-checks cleanly.

Limitations I would tell a client

- The YOLO labels come from the classical pipeline, not from people. The model learns to generalise and to handle blur, but in dense closed canopy some crowns are still merged. The Validation tab (click trees yourself, get precision/recall) is how to measure it on a new site.
- At 1 m per pixel a 3 m crown is 3 pixels. Canopy cover stays reliable; the crown count becomes a lower bound, and the app says so.
- Free satellite imagery has unknown capture dates, so heights and change over time need dated imagery.

What I would do next

- Hand-label 200–300 crowns on Odisha imagery and fine-tune on them; that is the single biggest accuracy lever.
- Train the high-accuracy model longer on a GPU and add a tiled SAHI-style inference for very large areas.
- Add dated imagery (Sentinel-2 / Planet) for change detection.